

# Neural-Driven Computing

Advanced Systems Engineering (ASE2026)

Paris Lodron University of Salzburg

Author: Bernhard Guillon

Date: 2026-04-29

# Research topics

- Have you ever wondered why we need a full stack of Operating System and a Browser to run AI workloads?
- Can't we just use LLMs to create "runnable" perceptrons instead of writing tons of code with it?
- Is it possible to use Perceptron-based computing as an operating system?

# Why this matters

- Current AI mostly requires heavy OS + Browser stack
- Neural networks are universal function approximators - why not use them directly?
- Perceptron-based computing could eliminate the "abstraction tax" of traditional OS
- Enables AI-native hardware without legacy baggage
- Could unlock orders-of-magnitude efficiency gains for specialized workloads

# Where we come from

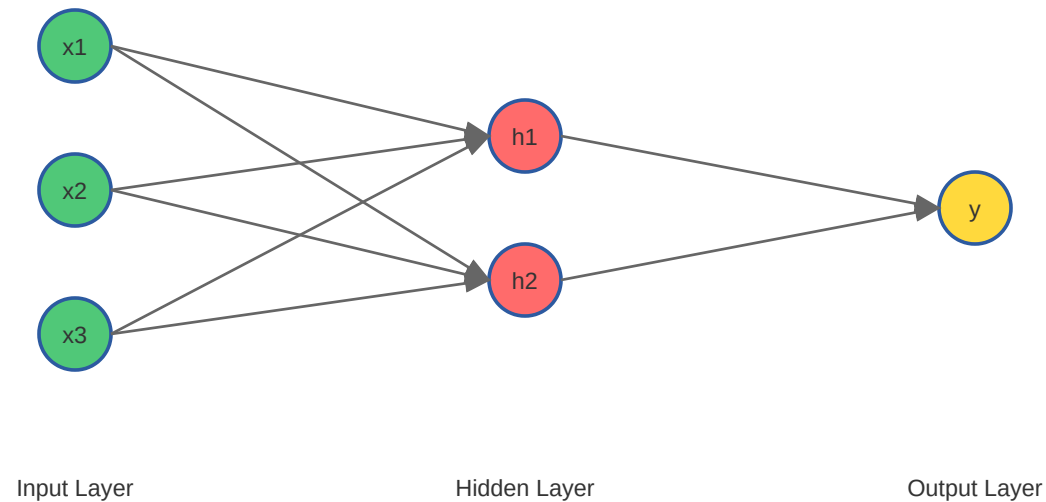
- 1940s-1950s: Punch Cards & Batch Processing
- 1960s: Mainframe OS & Time-Sharing
- 1970s-1980s: Personal Computers & GUIs - CP/M (1974) for microcomputers.
- 1990s: Windows & Linux - (GUI and multitasking)
- 2000s-2010s: Mobile OS with Browsers
- 2020s-now: AI all over but not for Operating Systems

# What an operating system does

- Abstracts the hardware
- Memory management
- I/O handling
- Allows multiple processes
- Multiple users
- Tries hard to separate processes and users from each other

# What a multilayer network does

Multilayer Perceptron



$$z_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1$$

$$h_1 = \text{ReLU}(z_1) \quad \text{ReLU}$$

$$z_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2$$

$$h_2 = \text{ReLU}(z_2)$$

$$z_3 = w_{31}h_1 + w_{32}h_2 + b_3$$

$$y = \text{softmax}(z_3) \quad \text{softmax}$$

# Approach

- RISC-V + Tensor OPs [✓]
- Emulator + RTL (system verilog) [✓]
- Assembler for the extensions [✓]
- Minimalistic bootloader [✓]
- Pytorch for training [✓]
- A "transpiler" for loadable and runnable artifact out of the trained network [✓]

# Simple character input [0..255] to framebuffer [20x20] output

```
##  
###  
# ##
```

```
##  
#   #  
#   ##  
# #### #  
##     ##  
##     ##  
##     ##
```



# Moveable character from [h,j,k,l] input in a [20x20] framebuffer

#

## Future work - squash mockup

Diagram illustrating a Pong game state:

- WALL (left):** The left boundary of the table.
- o <- ball:** The ball, represented by a small circle.
- PADDLE:** The paddle, represented by a vertical line segment.
- SCORE: 07:** The current score.

# Future work - pong over network

- Dual-emulator networking
- connect two emulator instances and exchange state/tokens each frame.
- multiplayer pong (left/right paddles, synchronized ball state).

# **Thank you**

**Thanks for your attention I hope you've enjoyed the presentation!**

**Feel free to ask me anything.**